Tugas Intelegensia Bisnis

Kategori 1: Data Preprocessing & ETL (Extract, Transform, Load)

Nama : **Muhammad Thifaal Dzaki**
NIM : **20230803103**

A. Soal

3. Standarisasi Format Tanggal dan Konversi Mata Uang

- **Kasus:** Data transaksi global perusahaan menggunakan berbagai format tanggal (DD-MM-YYYY vs YYYY/MM/DD) dan mata uang asing (USD, EUR, SGD).
- **Tugas Python:** Ubah seluruh kolom tanggal ke standar ISO (YYYY-MM-DD) dan konversikan seluruh mata uang ke Rupiah (IDR) berdasarkan tabel kurs harian yang tersedia.

B. Kode Program dan Penjabaran Fungsi (soal3_standarisasi_tanggal_kurs.py)

Berikut kode program yang saya buat. Kodenya saya bagi per fungsi sesuai urutan di dalam file, dan di bawah setiap bagian saya jelaskan kegunaan fungsi tersebut.

B.1 Docstring, Import, dan Konfigurasi

```
"""
Studi Kasus 3: Standarisasi Format Tanggal dan Konversi Mata Uang
----------------------------------------------------------------
Kasus : Data transaksi global memakai berbagai format tanggal
        (DD-MM-YYYY vs YYYY/MM/DD) dan mata uang asing (USD, EUR, SGD).
Tugas : Ubah seluruh kolom tanggal ke standar ISO (YYYY-MM-DD) dan
        konversikan seluruh mata uang ke Rupiah (IDR) berdasarkan tabel
        kurs harian yang tersedia.

Langkah:
  1. Muat data transaksi & tabel kurs (atau buat data contoh)
  2. Standarisasi tanggal -> ISO YYYY-MM-DD (deteksi format per baris)
  3. Konversi nominal -> IDR memakai kurs harian
     (jika kurs tanggal itu tidak ada, pakai kurs terakhir sebelumnya)
  4. Validasi & simpan hasil

Jalankan:
  python soal3_standarisasi_tanggal_kurs.py                      # data contoh
  python soal3_standarisasi_tanggal_kurs.py transaksi.csv kurs.csv # file sendiri

Format file sendiri:
  transaksi.csv : id_transaksi, tanggal, negara, mata_uang, nominal
  kurs.csv      : tanggal (YYYY-MM-DD), mata_uang, kurs_idr
"""

import os
import sys
from pathlib import Path

import numpy as np
import pandas as pd

# ---------------------------------------------------------------------------
# KONFIGURASI
# ---------------------------------------------------------------------------
FILE_TRANSAKSI = "transaksi_global.csv"
FILE_KURS = "kurs_harian.csv"
```

```
OUTPUT_CSV = "transaksi_global_idr.csv"
OUTPUT_EXCEL = "transaksi_global_idr.xlsx"

# Pola format tanggal yang dikenali: (regex, format strptime)
FORMAT_TANGGAL = [
    (r"^\d{2}-\d{2}-\d{4}$", "%d-%m-%Y"),   # DD-MM-YYYY
    (r"^\d{4}/\d{2}/\d{2}$", "%Y/%m/%d"),   # YYYY/MM/DD
    (r"^\d{4}-\d{2}-\d{2}$", "%Y-%m-%d"),   # sudah ISO
]

KURS_DASAR = {"USD": 16_300, "EUR": 17_700, "SGD": 12_600}  # untuk data contoh
NEGARA = {"USD": "Amerika Serikat", "EUR": "Jerman", "SGD": "Singapura",
          "IDR": "Indonesia"}
```

Bagian awal berisi docstring tentang studi kasus, langkah program, dan cara menjalankannya. Library yang dipakai yaitu **os**, **sys**, dan **Path** untuk folder, file, dan argumen, **numpy** untuk membuat data acak, serta **pandas** untuk mengolah tabel.

Di bagian konfigurasi ada **FORMAT_TANGGAL**, yaitu daftar pola tanggal yang bisa dikenali oleh program beserta cara membacanya, yaitu DD-MM-YYYY, YYYY/MM/DD, dan YYYY-MM-DD. Dengan cara ini, kalau nanti ada format baru cukup ditambahkan ke daftar ini. **KURS_DASAR** dan **NEGARA** dipakai saat membuat data contoh.

B.2 Fungsi buat_data_contoh()

```
# ---------------------------------------------------------------------------
# 1. MUAT DATA (atau buat data contoh)
# ---------------------------------------------------------------------------
def buat_data_contoh(path_trx: str, path_kurs: str, n: int = 60, seed: int = 42) -> None:
    rng = np.random.default_rng(seed)
    hari = pd.date_range("2026-09-01", "2026-09-14", freq="D")

    # Tabel kurs harian: hanya hari kerja (Sabtu/Minggu tidak ada kurs)
    hari_kerja = hari[hari.dayofweek < 5]
    kurs = pd.DataFrame([
        {"tanggal": t.strftime("%Y-%m-%d"), "mata_uang": mu,
         "kurs_idr": round(dasar * (1 + rng.normal(0, 0.004)), 2)}
        for t in hari_kerja for mu, dasar in KURS_DASAR.items()
    ])
    kurs.to_csv(path_kurs, index=False)

    # Transaksi dengan format tanggal campuran
    mata_uang = rng.choice(["USD", "EUR", "SGD", "IDR"], n, p=[0.35, 0.25, 0.25, 0.15])
    tgl = rng.choice(hari, n)
    nominal = [
        round(rng.uniform(100_000, 5_000_000), -3) if mu == "IDR"
        else round(rng.uniform(10, 500), 2)
        for mu in mata_uang
    ]
    teks_tgl = [
        pd.Timestamp(t).strftime("%d-%m-%Y" if i % 2 == 0 else "%Y/%m/%d")
        for i, t in enumerate(tgl)
    ]
    trx = pd.DataFrame({
        "id_transaksi": [f"TRX-{i:03d}" for i in range(1, n + 1)],
        "tanggal": teks_tgl,
        "negara": [NEGARA[mu] for mu in mata_uang],
        "mata_uang": mata_uang,
        "nominal": nominal,
    })
    trx.to_csv(path_trx, index=False)
    print(f"[INFO] Data contoh dibuat: {path_trx}, {path_kurs}")
```

Fungsi ini membuat data contoh jika file sumber belum ada. Tabel kurs dibuat hanya untuk hari kerja dengan **dayofweek < 5**, jadi hari Sabtu dan Minggu memang tidak punya kurs. Data

transaksinya berjumlah 60 baris dengan mata uang USD, EUR, SGD, dan IDR, dan tanggalnya ditulis bergantian dengan format DD-MM-YYYY dan YYYY/MM/DD supaya sesuai dengan kasus di soal. Keduanya disimpan ke CSV dengan **to_csv()**.

B.3 Fungsi muat_data()

```
92  def muat_data(path_trx: str, path_kurs: str) -> tuple[pd.DataFrame, pd.DataFrame]:
93      print(f"[INFO] Memuat transaksi: {path_trx}")
94      print(f"[INFO] Memuat kurs harian: {path_kurs}")
95      # dtype=str agar tanggal tidak ditebak otomatis oleh pandas
96      trx = pd.read_csv(path_trx, dtype={"tanggal": str})
97      kurs = pd.read_csv(path_kurs, dtype={"tanggal": str})
98      return trx, kurs
```

Fungsi ini membaca file transaksi dan file kurs. Kolom tanggal dibaca sebagai teks dengan **dtype={"tanggal": str}** supaya pandas tidak menebak formatnya sendiri.

B.4 Fungsi deteksi_format()

```
101 # ------------------------------------------------------------------------
102 # 2. STANDARISASI TANGGAL -> ISO (YYYY-MM-DD)
103 # ------------------------------------------------------------------------
104 def deteksi_format(teks: pd.Series) -> pd.Series:
105     """Label format tanggal tiap baris (untuk laporan)."""
106     label = pd.Series("TIDAK DIKENALI", index=teks.index)
107     for pola, fmt in FORMAT_TANGGAL:
108         label[teks.str.match(pola)] = fmt
109     return label
```

Fungsi ini mencatat format tanggal asli dari setiap baris. Setiap pola di **FORMAT_TANGGAL** dicocokkan dengan **str.match()**, dan baris yang cocok diberi label sesuai formatnya. Kalau tidak cocok dengan pola mana pun, labelnya "TIDAK DIKENALI". Hasilnya dipakai untuk laporan berapa banyak tanggal per format.

B.5 Fungsi ke_iso()

```
112 def ke_iso(teks: pd.Series) -> pd.Series:
113     """Parse tiap baris sesuai polanya; tidak memakai tebakan otomatis
114     agar 03-09-2026 tidak tertukar menjadi 9 Maret."""
115     teks = teks.astype(str).str.strip()
116     hasil = pd.Series(pd.NaT, index=teks.index, dtype="datetime64[ns]")
117     for pola, fmt in FORMAT_TANGGAL:
118         cocok = teks.str.match(pola)
119         hasil[cocok] = pd.to_datetime(teks[cocok], format=fmt, errors="coerce")
120     return hasil
```

Fungsi ini mengubah teks tanggal menjadi tipe tanggal. Teks dibersihkan dulu dari spasi, lalu setiap baris dicocokkan dengan polanya. Baris yang cocok dibaca dengan **pd.to_datetime()** memakai format yang sesuai. Karena format ditentukan sendiri dan tidak ditebak, tanggal seperti 03-09-2026 tetap terbaca 3 September, bukan 9 Maret.

B.6 Fungsi standarisasi_tanggal()

```
123 def standarisasi_tanggal(df: pd.DataFrame, kolom: str = "tanggal") -> pd.DataFrame:
124     df = df.copy()
125     df["tanggal_asli"] = df[kolom]
126     df["format_asli"] = deteksi_format(df[kolom].astype(str).str.strip())
127     df[kolom] = ke_iso(df[kolom])
128     gagal = df[kolom].isna().sum()
129     if gagal:
130         print(f"[PERINGATAN] {gagal} tanggal tidak dapat diparse:")
131         print(df.loc[df[kolom].isna(), ["id_transaksi", "tanggal_asli"]].to_string(index=False))
132     return df
```

Fungsi ini menjalankan proses standarisasi pada tabel transaksi. Tanggal asli disimpan dulu ke kolom **tanggal_asli**, formatnya dicatat di **format_asli** memakai **deteksi_format()**, lalu kolom **tanggal** diubah dengan **ke_iso()**. Kalau ada tanggal yang gagal dibaca, program menampilkan peringatan beserta ID transaksinya supaya bisa dicek.

B.7 Fungsi siapkan_kurs()

```
135  # ------------------------------------------------------------------------
136  # 3. KONVERSI MATA UANG -> IDR
137  # ------------------------------------------------------------------------
138  def siapkan_kurs(kurs: pd.DataFrame) -> pd.DataFrame:
139      kurs = kurs.copy()
140      kurs["tanggal"] = ke_iso(kurs["tanggal"])
141      kurs["mata_uang"] = kurs["mata_uang"].str.strip().str.upper()
142      return kurs.dropna(subset=["tanggal"]).sort_values("tanggal")
```

Fungsi ini merapikan tabel kurs sebelum dipakai. Tanggal kurs juga diubah dengan **ke_iso()**, kode mata uang dijadikan huruf besar, baris yang tanggalnya kosong dibuang, lalu tabel diurutkan berdasarkan tanggal. Pengurutan ini perlu karena **merge_asof** hanya bisa berjalan pada data yang sudah urut.

B.8 Fungsi konversi_ke_idr()

```
145  def konversi_ke_idr(df: pd.DataFrame, kurs: pd.DataFrame) -> pd.DataFrame:
146      df = df.copy()
147      df["mata_uang"] = df["mata_uang"].str.strip().str.upper()
148      df["_urut"] = np.arange(len(df))
149
150      # merge_asof: ambil kurs pada tanggal transaksi, atau kurs terakhir
151      # sebelumnya (mis. transaksi hari Sabtu/Minggu memakai kurs Jumat)
152      df = pd.merge_asof(
153          df.sort_values("tanggal"),
154          kurs.rename(columns={"tanggal": "tanggal_kurs"}).sort_values("tanggal_kurs"),
155          left_on="tanggal", right_on="tanggal_kurs",
156          by="mata_uang", direction="backward",
157      ).sort_values("_urut").drop(columns="_urut")
158
159      # IDR tidak perlu dikonversi
160      idr = df["mata_uang"] == "IDR"
161      df.loc[idr, "kurs_idr"] = 1.0
162      df.loc[idr, "tanggal_kurs"] = df.loc[idr, "tanggal"]
163
164      df["nominal_idr"] = (df["nominal"] * df["kurs_idr"]).round(0)
165      return df.reset_index(drop=True)
```

Fungsi ini mengubah nominal transaksi ke Rupiah. Saya menambahkan kolom bantu **_urut** untuk menyimpan urutan asli transaksi, karena data harus diurutkan berdasarkan tanggal sebelum digabung dan nantinya dikembalikan ke urutan semula.

Penggabungan dengan kurs memakai **pd.merge_asof()** dengan **by="mata_uang"** dan **direction="backward"**. Setiap transaksi mendapat kurs mata uang yang sama pada tanggal transaksi, dan kalau tanggal itu tidak ada kurs maka dipakai kurs hari kerja terakhir sebelumnya. Tanggal kurs yang dipakai dicatat di kolom **tanggal_kurs**. Untuk transaksi IDR, **kurs_idr** diisi 1. Terakhir **nominal_idr** dihitung dari **nominal** dikali **kurs_idr**.

B.9 Fungsi validasi()

```
168  # ------------------------------------------------------------------------
169  # 4. VALIDASI & SIMPAN
170  # ------------------------------------------------------------------------
171  def validasi(df: pd.DataFrame) -> None:
172      assert df["tanggal"].notna().all(), "Masih ada tanggal yang gagal diparse!"
```

```
    tanpa_kurs = df[df["kurs_idr"].isna()]
    assert tanpa_kurs.empty, (
        "Kurs tidak tersedia untuk:\n"
        + tanpa_kurs[["id_transaksi", "tanggal", "mata_uang"]].to_string(index=False)
    )
    assert (df["nominal_idr"] > 0).all(), "Ada nominal IDR <= 0!"
```

Fungsi ini mengecek hasil akhir. Tidak boleh ada tanggal yang gagal dibaca, setiap transaksi harus mendapatkan kurs, dan semua **nominal_idr** harus lebih dari 0. Kalau ada transaksi yang tidak dapat kurs, daftar transaksinya ikut ditampilkan di pesan kesalahan.

B.10 Fungsi susun_output()

```
def susun_output(df: pd.DataFrame) -> pd.DataFrame:
    out = df[["id_transaksi", "tanggal_asli", "format_asli", "tanggal", "negara",
              "mata_uang", "nominal", "tanggal_kurs", "kurs_idr", "nominal_idr"]].copy()
    out["tanggal"] = out["tanggal"].dt.strftime("%Y-%m-%d")
    out["tanggal_kurs"] = out["tanggal_kurs"].dt.strftime("%Y-%m-%d")
    return out
```

Fungsi ini memilih kolom yang akan disimpan dan mengurutkannya. Kolom **tanggal** dan **tanggal_kurs** diubah menjadi teks berformat **YYYY-MM-DD** dengan **dt.strftime()**, sesuai standar ISO yang diminta soal.

B.11 Fungsi simpan()

```
def simpan(out: pd.DataFrame, ringkasan: pd.DataFrame) -> None:
    out.to_csv(OUTPUT_CSV, index=False)
    with pd.ExcelWriter(OUTPUT_EXCEL, engine="openpyxl") as writer:
        out.to_excel(writer, sheet_name="Transaksi IDR", index=False)
        ringkasan.to_excel(writer, sheet_name="Ringkasan Mata Uang")
    print(f"\n[OK] Disimpan: {OUTPUT_CSV}, {OUTPUT_EXCEL}")
```

Fungsi ini menyimpan hasil ke CSV dengan **to_csv()** dan ke Excel dengan **pd.ExcelWriter**. File Excel berisi dua sheet, yaitu Transaksi IDR dan Ringkasan Mata Uang.

B.12 Fungsi main()

```
# ---------------------------------------------------------------------------
# MAIN
# ---------------------------------------------------------------------------
def main() -> None:
    os.chdir(Path(__file__).resolve().parent)  # baca/simpan file di folder ini
    path_trx = sys.argv[1] if len(sys.argv) > 1 else FILE_TRANSAKSI
    path_kurs = sys.argv[2] if len(sys.argv) > 2 else FILE_KURS
    if not (Path(path_trx).exists() and Path(path_kurs).exists()):
        print("[INFO] File sumber tidak ditemukan -> membuat data contoh.")
        buat_data_contoh(path_trx, path_kurs)

    trx, kurs = muat_data(path_trx, path_kurs)

    df = standarisasi_tanggal(trx)
    print("\n=== FORMAT TANGGAL YANG TERDETEKSI ===")
    print(df["format_asli"].value_counts().to_string())

    df = konversi_ke_idr(df, siapkan_kurs(kurs))
    validasi(df)
    out = susun_output(df)

    pakai_kurs_sebelumnya = (out["tanggal"] != out["tanggal_kurs"]).sum()
    print(f"\nTransaksi memakai kurs hari sebelumnya (akhir pekan/libur): "
          f"{pakai_kurs_sebelumnya}")

    print("\n=== CONTOH HASIL (10 baris pertama) ===")
    print(out.head(10).to_string(index=False))

    ringkasan = (out.groupby("mata_uang")
```

```
                        .agg(jumlah_transaksi=("id_transaksi", "count"),
                             total_nominal_asli=("nominal", "sum"),
                             rata_rata_kurs=("kurs_idr", "mean"),
                             total_idr=("nominal_idr", "sum"))
                        .round(2))
        print("\n=== RINGKASAN PER MATA UANG ===")
        print(ringkasan.to_string())
        print(f"\nTotal seluruh transaksi: Rp {out['nominal_idr'].sum():,.0f}")

        simpan(out, ringkasan)
```

Fungsi **main()** mengatur jalannya program. Folder kerja dipindahkan dengan **os.chdir()**, nama file bisa diberikan lewat **sys.argv**, dan kalau filenya tidak ada maka dibuat data contoh dengan **buat_data_contoh()**.

Setelah data dibaca, tanggal distandarkan dengan **standarisasi_tanggal()**, nominal dikonversi dengan **konversi_ke_idr()**, hasilnya dicek dengan **validasi()**, lalu disusun dengan **susun_output()**. Program juga menghitung berapa transaksi yang memakai kurs hari sebelumnya dan membuat ringkasan per mata uang dengan **groupby()**. Hasil akhirnya ditampilkan lalu disimpan dengan **simpan()**.

B.13 Bagian if __name__

```
if __name__ == "__main__":
    main()
```

Baris terakhir ini membuat fungsi **main()** hanya dijalankan kalau file dibuka langsung lewat terminal. Jadi kalau file ini di-import dari program lain, prosesnya tidak langsung berjalan.

C. Hasil / Jawaban

Program saya jalankan dengan perintah **python soal3_standarisasi_tanggal_kurs.py** dari folder tempat file program berada, dan output yang muncul di terminal adalah sebagai berikut.

```
[INFO] Memuat transaksi: transaksi_global.csv
[INFO] Memuat kurs harian: kurs_harian.csv

=== FORMAT TANGGAL YANG TERDETEKSI ===
format_asli
%d-%m-%Y    30
%Y/%m/%d    30

Transaksi memakai kurs hari sebelumnya (akhir pekan/libur): 13

=== CONTOH HASIL (10 baris pertama) ===
id_transaksi tanggal_asli format_asli    tanggal          negara mata_uang    nominal tanggal_kurs  kurs_idr  nominal_idr
     TRX-001   14-09-2026    %d-%m-%Y 2026-09-14       Singapura       SGD     296.21   2026-09-14  12621.71    3738677.0
     TRX-002   2026/09/03    %Y/%m/%d 2026-09-03       Indonesia       IDR 3284000.00   2026-09-03     1.00    3284000.0
     TRX-003   07-09-2026    %d-%m-%Y 2026-09-07 Amerika Serikat       USD      51.38   2026-09-07  16304.31     837715.0
     TRX-004   2026/09/10    %Y/%m/%d 2026-09-10          Jerman       EUR     213.75   2026-09-10  17786.56    3801877.0
     TRX-005   07-09-2026    %d-%m-%Y 2026-09-07          Jerman       EUR      30.39   2026-09-07  17779.81     540328.0
     TRX-006   2026/09/07    %Y/%m/%d 2026-09-07 Amerika Serikat       USD     252.06   2026-09-07  16304.31    4109664.0
     TRX-007   03-09-2026    %d-%m-%Y 2026-09-03 Amerika Serikat       USD     171.63   2026-09-03  16308.34    2799000.0
     TRX-008   2026/09/06    %Y/%m/%d 2026-09-06          Jerman       EUR      80.82   2026-09-04  17762.26    1435546.0
     TRX-009   04-09-2026    %d-%m-%Y 2026-09-04 Amerika Serikat       USD      60.67   2026-09-04  16244.38     985547.0
     TRX-010   2026/09/05    %Y/%m/%d 2026-09-05       Singapura       SGD     297.95   2026-09-04  12639.20    3765850.0

=== RINGKASAN PER MATA UANG ===
           jumlah_transaksi  total_nominal_asli  rata_rata_kurs   total_idr
mata_uang
EUR                      17             4497.23        17722.84  79718936.0
IDR                       2          4512000.00            1.00   4512000.0
SGD                      19             5362.65        12613.11  67648380.0
USD                      22             4476.10        16288.20  72889652.0

Total seluruh transaksi: Rp 224,768,968

[OK] Disimpan: transaksi_global_idr.csv, transaksi_global_idr.xlsx
```

Gambar 1. Output program di terminal.

Program mendeteksi 30 tanggal berformat DD-MM-YYYY dan 30 tanggal berformat YYYY/MM/DD, dan semuanya berhasil diubah ke format ISO. Ada 13 transaksi yang terjadi di akhir pekan sehingga memakai kurs hari kerja sebelumnya, contohnya TRX-008 pada hari Minggu 2026-09-06 dan TRX-010 pada hari Sabtu 2026-09-05 yang sama-sama memakai kurs tanggal 2026-09-04.

Sebagai contoh konversi, TRX-007 sebesar 171,63 USD dikali 16.308,34 menghasilkan Rp 2.799.000. Jika dilihat per mata uang, EUR ada 17 transaksi dengan total Rp 79.718.936, IDR 2 transaksi dengan total Rp 4.512.000, SGD 19 transaksi dengan total Rp 67.648.380, dan USD 22 transaksi dengan total Rp 72.889.652. Total semua transaksi adalah Rp 224.768.968. Hasil disimpan ke **transaksi_global_idr.csv** dan **transaksi_global_idr.xlsx**.

id_transaksi	tanggal_asli	format_asli	tanggal	negara	mata_uang	nominal	tanggal_kurs	kurs_idr	nominal_idr
TRX-001	14-09-2026	%d-%m-%Y	2026-09-14	Singapura	SGD	296.21	2026-09-14	12621.71	3738677.0
TRX-002	2026/09/03	%Y/%m/%d	2026-09-03	Indonesia	IDR	3284000.0	2026-09-03	1.0	3284000.0
TRX-003	07-09-2026	%d-%m-%Y	2026-09-07	Amerika Serikat	USD	51.38	2026-09-07	16304.31	837715.0
TRX-004	2026/09/10	%Y/%m/%d	2026-09-10	Jerman	EUR	213.75	2026-09-10	17786.56	3801877.0
TRX-005	07-09-2026	%d-%m-%Y	2026-09-07	Jerman	EUR	30.39	2026-09-07	17779.81	540328.0
TRX-006	2026/09/07	%Y/%m/%d	2026-09-07	Amerika Serikat	USD	252.06	2026-09-07	16304.31	4109664.0
TRX-007	03-09-2026	%d-%m-%Y	2026-09-03	Amerika Serikat	USD	171.63	2026-09-03	16308.34	2799000.0
TRX-008	2026/09/06	%Y/%m/%d	2026-09-06	Jerman	EUR	80.82	2026-09-04	17762.26	1435546.0
TRX-009	04-09-2026	%d-%m-%Y	2026-09-04	Amerika Serikat	USD	60.67	2026-09-04	16244.38	985547.0
TRX-010	2026/09/05	%Y/%m/%d	2026-09-05	Singapura	SGD	297.95	2026-09-04	12639.2	3765850.0
TRX-011	10-09-2026	%d-%m-%Y	2026-09-10	Jerman	EUR	93.59	2026-09-10	17786.56	1664644.0
TRX-012	2026/09/09	%Y/%m/%d	2026-09-09	Singapura	SGD	463.31	2026-09-09	12590.68	5833388.0
TRX-013	09-09-2026	%d-%m-%Y	2026-09-09	Singapura	SGD	294.72	2026-09-09	12590.68	3710725.0
TRX-014	2026/09/06	%Y/%m/%d	2026-09-06	Amerika Serikat	USD	179.97	2026-09-04	16244.38	2923501.0
TRX-015	14-09-2026	%d-%m-%Y	2026-09-14	Singapura	SGD	299.55	2026-09-14	12621.71	3780833.0
TRX-016	2026/09/02	%Y/%m/%d	2026-09-02	Singapura	SGD	21.17	2026-09-02	12534.37	265353.0
TRX-017	05-09-2026	%d-%m-%Y	2026-09-05	Jerman	EUR	479.69	2026-09-04	17762.26	8520378.0
TRX-018	2026/09/02	%Y/%m/%d	2026-09-02	Amerika Serikat	USD	246.33	2026-09-02	16361.32	4030284.0
TRX-019	05-09-2026	%d-%m-%Y	2026-09-05	Singapura	SGD	393.54	2026-09-04	12639.2	4974031.0
TRX-020	2026/09/14	%Y/%m/%d	2026-09-14	Amerika Serikat	USD	50.54	2026-09-14	16323.83	825006.0
TRX-021	06-09-2026	%d-%m-%Y	2026-09-06	Amerika Serikat	USD	248.46	2026-09-04	16244.38	4036079.0
TRX-022	2026/09/13	%Y/%m/%d	2026-09-13	Amerika Serikat	USD	250.45	2026-09-11	16272.07	4075340.0
TRX-023	07-09-2026	%d-%m-%Y	2026-09-07	Singapura	SGD	469.53	2026-09-07	12623.56	5927140.0
TRX-024	2026/09/10	%Y/%m/%d	2026-09-10	Singapura	SGD	290.15	2026-09-10	12592.21	3653630.0
TRX-025	07-09-2026	%d-%m-%Y	2026-09-07	Singapura	SGD	242.01	2026-09-07	12623.56	3055028.0

Gambar 2. Isi file output transaksi_global_idr.csv (25 baris pertama dari 60 baris).

Kesimpulan:

Semua tanggal berhasil diubah ke format ISO **YYYY-MM-DD** dengan cara mencocokkan pola setiap baris, sehingga hari dan bulan tidak tertukar dan format baru mudah ditambahkan. Konversi ke Rupiah memakai **merge_asof** per mata uang, jadi transaksi di akhir pekan tetap

mendapat kurs dari hari kerja sebelumnya, dan kolom **tanggal_kurs** menunjukkan kurs tanggal berapa yang dipakai. Dengan adanya pengecekan di akhir, bisa dipastikan tidak ada tanggal yang gagal dibaca atau transaksi yang tidak mendapat kurs.